CS 432: Databases

Assignment 02
Module A

Track 01

by
Nitin Bansal
Devvrat Hans
Sailaja Mudu
Thipparapu Rushitha
Vadithya Harsha Vardhan Nayak
of Group 21

Bachelor of Technology,
Indian Institute of Technology Gandhinagar

Contents

Introduction ……...…….3

Methodology……...…..3

1. **B+ Tree Logic…….…….…….…….6**
2. **Brute force logic….……..7**
3. **DB Manager logic…..…..…….……7**
4. **Table implementation logic….……..11**

Performance Analysis…….……..……..13

1. **Analysis of Benchmark Results……....15**
2. **Graphical Analysis…….16**

B+ Tree Visualisation……...….17

DB Manager Implementation Demonstration…….16

Conclusion……..18

Team Contribution Table……..19

Github Repository Link

- *https://github.com/HARSHA219219/DBMS_Assignment-2_Module_A*

Video Link:

- *https://drive.google.com/drive/folders/1vy1oG43t-MHgb68K1NBUGmmTZRMDDTjd?usp=drive*

Module A: Lightweight DBMS with B+ Tree Index.

Introduction

Efficient data storage, retrieval, and management are fundamental challenges in modern database systems, especially as the volume of data continues to grow. Traditional data storage methods, such as linear data structures, suffer from poor scalability, as operations like search, deletion, and range queries require scanning the entire dataset. This results in time complexities of O(n), making such approaches inefficient for large-scale applications.

To address these limitations, this module focuses on implementing a B+ Tree-based indexing system as part of a lightweight Database Management System (DBMS). The B+ Tree is a self-balancing tree structure widely used in database systems and file systems due to its ability to maintain sorted data and support efficient operations in logarithmic time. Unlike simple tree structures, the B+ Tree stores all actual data in leaf nodes and maintains a linked structure between these leaves, enabling fast sequential access and efficient range queries.

In this module, a B+ Tree is implemented from scratch in Python, supporting core operations such as insertion, deletion, exact search, update, and range queries. Additionally, a brute-force database implementation is developed using a linear data structure to serve as a baseline for comparison.

To evaluate performance, a comprehensive benchmarking framework is used to compare both approaches across multiple operations, including insertion, search, deletion, range queries, and mixed workloads. The results demonstrate how structured indexing using B+ Trees significantly improves efficiency and scalability compared to linear methods.

Overall, this project highlights the importance of advanced data structures in database design and provides practical insights into how indexing mechanisms enhance performance in real-world systems.

Methodology

The project follows a structured approach to design, implement, and evaluate a lightweight database system using a B+ Tree as the primary indexing mechanism. The methodology is divided into the following stages:

1. System Design

The system is designed in a modular manner with clearly separated components:

- B+ Tree Module for efficient indexing
- BruteForceDB Module for baseline comparison
- DB Manager Module for table-level operations
- Performance Analyzer Module for benchmarking

This modular design ensures scalability, maintainability, and ease of testing.

2. Data Structure Implementation

The core of the system is the B+ Tree, which is implemented from scratch with the following features:

- Balanced tree structure with configurable order ttt
- Support for insertion, deletion, search, update, and range queries
- Linked leaf nodes for efficient sequential access

Alongside this, a brute-force implementation is developed using a linear data structure to provide a performance baseline.

3. DBMS Layer Development

A lightweight DBMS layer is built on top of the B+ Tree:

- Tables are created with optional schema definitions
- CRUD operations (Create, Read, Update, Delete) are supported
- Range queries and bulk insert operations are implemented
- Input validation and error handling are included

This layer simulates real-world database behaviour and integrates the indexing structure effectively.

4. Performance Benchmarking

To evaluate the efficiency of the B+ Tree, a performance analysis framework is developed:

- Datasets of varying sizes (5,000 to 50,000 records) are generated
- The following operations are measured:
    - Insertion
    - Search
    - Deletion
    - Range queries
    - Random mixed operations
    - Memory usage

Execution time is recorded using high-resolution timers, and memory usage is estimated using system-level functions.

5. Experimental Evaluation

Both the B+ Tree and Brute Force implementations are tested under identical conditions:

- Same dataset inputs
- Same sequence of operations
- Multiple runs to ensure consistency

The results are tabulated and compared to analyse performance differences.

Implementation Details:

B+ Tree Logic

The B+ Tree is implemented as a self-balancing tree structure with order t=3, ensuring efficient insertion, deletion, and search operations. The implementation consists of two main classes: BPlusTreeNode and BPlusTree.

1. Node Structure

Each node in the B+ Tree contains the following components:

- keys: A sorted list of keys
- children: Pointers to child nodes (used in internal nodes)
- values: Stores corresponding values (used only in leaf nodes)
- leaf: Boolean flag indicating whether the node is a leaf
- next: Pointer to the next leaf node

The next pointer plays a crucial role in enabling efficient range queries by linking all leaf nodes in a sequential manner.

2. Tree Initialisation

The B+ Tree is initialised with:

- A root node (initially a leaf)
- Order t=3, which defines:
    - Maximum keys per node = $2t-1$
    - Minimum keys per node = $t-1$

3. Search Operation

The search operation starts at the root and recursively traverses the tree:

- If the node is a leaf, a linear search is performed within the keys
- If the node is internal, the appropriate child pointer is selected based on key comparison

This ensures a time complexity of **O(logn)**

4. Insertion Operation

Insertion is performed in two stages:

Step 1: Root Check

- If the root node is full, it is split into two nodes

- A new root is created

Step 2: Insert into Non-Full Node

- For leaf nodes:
  - Keys and values are inserted in sorted order
- For internal nodes:
  - Recursively descend into the appropriate child
  - Split the child if it is full before insertion

5. Node Splitting

The _split_child function handles node overflow:

- The middle index is calculated as $t-1$
- For leaf nodes:
  - Keys and values are split into two nodes
  - The first key of the new node is promoted to the parent
  - The next pointer is updated to maintain leaf linkage
- For internal nodes:
  - The middle key is promoted to the parent
  - Children are redistributed accordingly

This ensures the tree remains balanced after insertion.

6. Deletion Operation

Deletion is handled carefully to maintain tree balance:

Step 1: Search for Key

- If the key does not exist, deletion is aborted

Step 2: Delete from Leaf

- If the key is in a leaf node, it is directly removed

Step 3: Handle Underflow

If a node has fewer than $t-1$ keys:

- Borrow from the left sibling if possible
- Otherwise, borrow from the right sibling
- If borrowing is not possible, merge nodes

Step 4: Root Adjustment

- If the root becomes empty, it is replaced by its child

7. Borrowing and Merging

- Borrowing:
  - Transfers a key from a sibling node
  - Updates parent keys accordingly
- Merging:

        - Combines two sibling nodes
        - Updates parent structure

    For leaf nodes, merging also ensures that the next pointer is correctly updated.

8. **<u>Update Operation</u>**

The update operation:

- Traverses to the appropriate leaf node
- Modifies the value corresponding to the given key

9. **<u>Range Query</u>**

The range query is one of the most important features:

- The tree is traversed to locate the starting leaf node
- Keys are collected sequentially using the next pointer
- Traversal stops once the end key is exceeded

This approach avoids scanning the entire dataset and significantly improves performance.

10. **<u>Traversal (Get All Records)</u>**
    - Starts from the leftmost leaf node
    - Iterates through all leaf nodes using next pointers
    - Collects all key-value pairs

<u>Bruteforce logic</u>

The Brute Force database is implemented using a simple linear data structure that stores records as key-value pairs. Unlike the B+ Tree, it does not use any indexing or hierarchical organisation, and all operations are performed through sequential traversal.

1. Data Storage

- Records are stored in a list as tuples of the form (key, value)
- No sorting or indexing is maintained
- Data is stored in the order of insertion

2. Insertion

- A new key-value pair is added to the end of the list using an append operation
- No checks for ordering or balancing are performed

3. Search

- The list is traversed sequentially from the beginning
- Each key is compared with the target key
- If a match is found, the corresponding value is returned
- If no match is found, None is returned

4. Deletion

- The list is traversed to locate the target key
- Once found, the corresponding element is removed

- The function returns True if deletion is successful, otherwise False

5. Range Query

- All elements in the list are scanned
- Records with keys within the specified range are collected
- The result is returned as a list of key-value pairs

6. Characteristics

- No indexing or structure is maintained
- All operations except insertion require full traversal
- Time complexity:
    - Insert → O(1)
    - Search → O(n)
    - Delete → O(n)
    - Range Query → O(n)

7. **Role in the System**

The Brute Force implementation is used as a baseline for comparison. It helps demonstrate how the absence of indexing leads to poor performance, especially for large datasets, thereby highlighting the efficiency of the B+ Tree.

DB Manager logic

The DB Manager acts as an abstraction layer that provides database-like functionality on top of the underlying data structures. It manages multiple tables and facilitates operations such as creation, deletion, and record manipulation using the B+ Tree as the indexing mechanism.

1. **Table Management**

- Tables are stored in a dictionary, where the table name acts as the key
- Each table is associated with a Table object

Supported operations include:

- Create Table: Initialises a new table with optional schema and indexing parameters
- Drop Table: Removes an existing table from the system
- List Tables: Returns all available table names
- Get Table: Retrieves a specific table instance

Validation is performed to prevent duplicate table creation and access to non-existent tables.

2. CRUD Operations

The DB Manager provides an interface for performing basic database operations:

- Insert: Adds a new record into the specified table using the underlying B+ Tree
- Search: Retrieves a record based on the given key
- Update: Modifies an existing record; raises an error if the key does not exist
- Delete: Removes a record from the table; ensures the key exists before deletion

All operations are delegated to the corresponding table methods, ensuring consistency and modularity.

3. Query Operations

- Range Query: Retrieves all records within a specified key range
    - Validates that the start key is not greater than the end key
    - Utilizes the B+ Tree for efficient retrieval
- Get All Records: Returns all stored records in sorted order

4. Bulk Operations

- Bulk Insert: Allows insertion of multiple records at once
- Performs validation to:
    - Ensure no duplicate keys exist within the input
    - Prevent insertion of keys that already exist in the table

This ensures data integrity during batch operations.

5. Metadata and Utility Functions

The DB Manager also provides utility functions:

- Table Size: Returns the number of records in a table
- Minimum Key: Retrieves the smallest key
- Maximum Key: Retrieves the largest key
- Count: Returns the total number of records

6. Error Handling

- Raises appropriate exceptions for:
    - Invalid table access
    - Duplicate table creation
    - Missing keys during update or delete
    - Invalid range queries

This ensures robustness and reliability of the system.

7. Role in the System

The DB Manager integrates all components into a unified system by acting as an interface between the user and the underlying data structures. It enables the B+ Tree to function as part of a complete database system rather than as an isolated data structure.

Table implementation logic

The Table class acts as an intermediate layer between the DB Manager and the B+ Tree. It represents a single database table and uses the B+ Tree as its underlying indexing structure to store and manage records efficiently.

1. Data Structure

- Each table is associated with:

    - A name identifying the table
    - A B+ Tree index for storing key-value pairs
    - A record count to track the number of entries
- The B+ Tree serves as the primary storage and indexing mechanism for all operations.

2. Insertion

- A new record is inserted using the B+ Tree index
- Validation is performed to:
    - Ensure the key is not None
    - Prevent duplicate keys
- On successful insertion:
    - The record is added to the B+ Tree
    - The record count is incremented

3. Search

- Retrieves a record using the B+ Tree search operation
- Ensures the key is valid before performing the search
- Returns the corresponding value if found

4. Update

- Updates the value associated with a given key
- Delegates the operation to the B+ Tree
- Returns a boolean indicating success or failure

5. Deletion

- Checks if the key exists before attempting deletion
- Removes the record using the B+ Tree delete operation
- Decrements the record count upon successful deletion

6. Range Query

- Retrieves records within a specified key range
- Validates input keys to ensure correctness
- Uses the B+ Tree range query for efficient retrieval

7. Data Retrieval

- Get All Records:
    - Retrieves all records in sorted order
    - Uses sequential traversal of leaf nodes

8. Utility Functions

The table provides additional helper operations:

- Size: Returns the total number of records
- Clear: Resets the table by reinitialising the B+ Tree
- Minimum Key: Retrieves the smallest key

- Maximum Key: Retrieves the largest key
- Count Records: Returns total record count

9. Error Handling

- Ensures keys are not None
- Prevents duplicate insertions
- Validates range query inputs

This improves reliability and maintains data integrity.

10. Role in the System

The Table class acts as a bridge between the DB Manager and the B+ Tree. It encapsulates all record-level operations while leveraging the efficiency of the B+ Tree, enabling the system to behave like a structured database.

Performance Analysis

To evaluate the performance of the implemented system, a series of benchmarking experiments were conducted on datasets of varying sizes, ranging from 5,000 to 50,000 records. The goal of these experiments is to compare the efficiency of the B+ Tree and the Brute Force approaches across different operations.

The results are presented in the form of a table for the following operations:

- Insertion
- Search
- Deletion
- Range Query
- Random Workload
- Memory Usage

Benchmark Results (seconds / bytes)

Size	Insert B+Tree	Insert Brute	Search B+Tree	Search Brute	Delete B+Tree	Delete Brute	Range B+Tree	Range Brute	Random B+Tree	Random Brute	Memory B+Tree	Memory Brute
5000	0.060949	0.002450	0.000417	0.013601	0.001309	0.032389	0.000287	0.000682	0.022820	0.229704	599896	321880
10000	0.099815	0.007582	0.000450	0.031991	0.001130	0.049364	0.000696	0.000893	0.055488	0.976262	1217816	645176
15000	0.130438	0.010523	0.000606	0.039850	0.001341	0.092583	0.001054	0.002394	0.103019	2.837309	1797816	961432
20000	0.245940	0.016179	0.000549	0.088697	0.001888	0.149613	0.001236	0.003040	0.139946	4.894552	2412968	1293016
25000	0.323565	0.013035	0.000576	0.105866	0.001781	0.220076	0.001253	0.004817	0.180723	7.703111	3025400	1619064
30000	0.402137	0.010220	0.000596	0.137070	0.001599	0.219032	0.002061	0.006209	0.230966	12.339763	3615928	1926488
35000	0.476490	0.255743	0.000591	0.152002	0.001739	0.306311	0.002581	0.006528	0.262932	16.459019	4244528	2272024
40000	0.570930	0.019420	0.000575	0.207036	0.001665	0.311052	0.002407	0.008022	0.483921	22.664206	4822016	2591064
45000	0.640418	0.021625	0.000601	0.221299	0.001707	0.413491	0.003595	0.010303	0.373304	29.000677	5438536	2914968
50000	0.888438	0.328363	0.000790	0.333020	0.001940	0.568033	0.004042	0.012169	0.475113	35.220374	6032752	3244376

Analysis of Benchmark Results

The benchmark results clearly highlight the performance differences between the B+ Tree and the Brute Force approach across various operations as the dataset size increases.

1. Insertion Analysis

From the table, it can be observed that the Brute Force approach consistently performs faster for insertion compared to the B+ Tree.

- For example, at 50,000 records:
    - B+ Tree: 0.8884 s
    - Brute Force: 0.3283 s

This is because Brute Force uses a simple append operation with constant time complexity O(1), whereas the B+ Tree requires additional operations such as node splitting and balancing. As a result, insertion in the B+ Tree incurs extra overhead.

2. Search Analysis

The B+ Tree significantly outperforms the Brute Force approach in search operations.

- At 50,000 records:
    - B+ Tree: 0.00079 s
    - Brute Force: 0.33302 s

This large difference occurs because:

- B+ Tree performs search in O(logn) time
- Brute Force performs search in O(n) time

As the dataset grows, the gap between the two approaches increases rapidly.

3. Deletion Analysis

The deletion performance also favours the B+ Tree.

- At 50,000 records:
    - B+ Tree: 0.00194 s
    - Brute Force: 0.56803 s

Even though B+ Tree deletion involves balancing operations such as borrowing and merging, it remains much faster than Brute Force, which requires a full traversal and shifting of elements.

4. Range Query Analysis

The B+ Tree performs more efficiently in range queries.

- At 50,000 records:
    - B+ Tree: 0.00404 s
    - Brute Force: 0.01217 s

This is due to the linked leaf node structure in the B+ Tree, which allows sequential traversal of only relevant data. In contrast, the Brute Force approach scans the entire dataset.

5. Random Workload Analysis

The most significant difference is observed in random operations:

- At 50,000 records:
    - B+ Tree: 0.4751 s
    - Brute Force: 35.2203 s

This demonstrates that:

- The B+ Tree handles mixed operations efficiently
- The Brute Force approach becomes extremely slow due to repeated linear scans

6. Memory Usage Analysis

The B+ Tree consumes more memory compared to the Brute Force approach.

- At 50,000 records:
    - B+ Tree: ~6.03 MB
    - Brute Force: ~3.24 MB

This is because the B+ Tree stores additional structural information such as nodes, child pointers, and leaf linkages. However, this extra memory usage enables significantly better performance.

7. Overall Interpretation

- The B+ Tree provides superior performance in:
    - Search
    - Deletion
    - Range queries
    - Mixed workloads
- The Brute Force approach is only advantageous for:
    - Simple insertion
- As the dataset size increases:
    - B+ Tree scales efficiently
    - Brute Force performance degrades rapidly

Graphical Analysis

1. **Graphical Analysis of Insertion Time Performance Comparison :**

The insertion performance graph compares the time required to insert records into the B+ Tree and the Brute Force data structure as the dataset size increases.

From the graph, it is observed that:

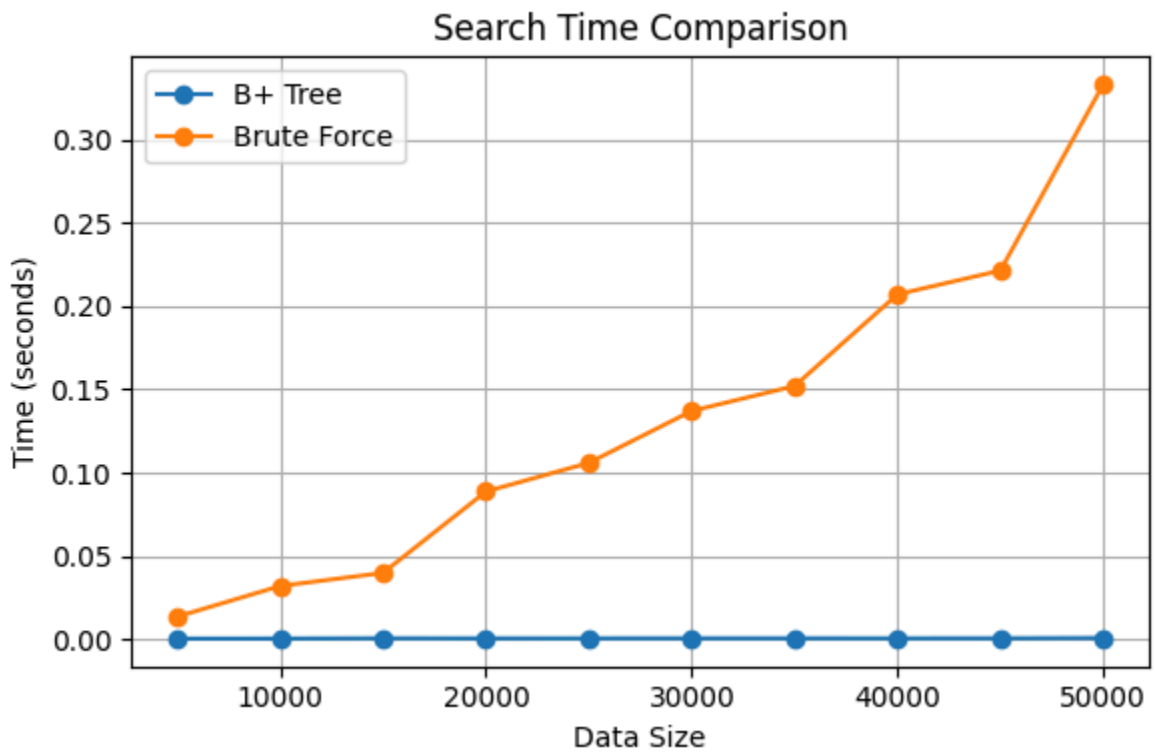
- The B+ Tree insertion time increases steadily with the growth in dataset size. This is due to the overhead of maintaining the tree structure, including node splitting and rebalancing operations. As more keys are inserted, these structural adjustments contribute to higher insertion time.
- The Brute Force approach consistently shows lower insertion time compared to the B+ Tree. This is because insertion is performed using a simple append operation, which has constant time complexity O(1) and does not require any structural maintenance.
- Although the Brute Force insertion is faster, the graph shows irregular spikes (e.g., around 35,000 and 50,000 records). These fluctuations may be due to system-level factors such as memory allocation or runtime overhead during execution.
- In contrast, the B+ Tree exhibits a smooth and predictable growth pattern, indicating stable and consistent performance despite increasing data size.

Key Insight

While the B+ Tree has a higher insertion time due to its balancing operations, this cost is justified as it enables significantly faster search, deletion, and range query operations. Therefore, the slightly higher insertion overhead is a trade-off for overall system efficiency.

2. **Graphical analysis of Search Time Performance Comparison:**

The search performance graph compares the time required to retrieve records from the B+ Tree and the Brute Force data structure as the dataset size increases.



From the graph, it is observed that:

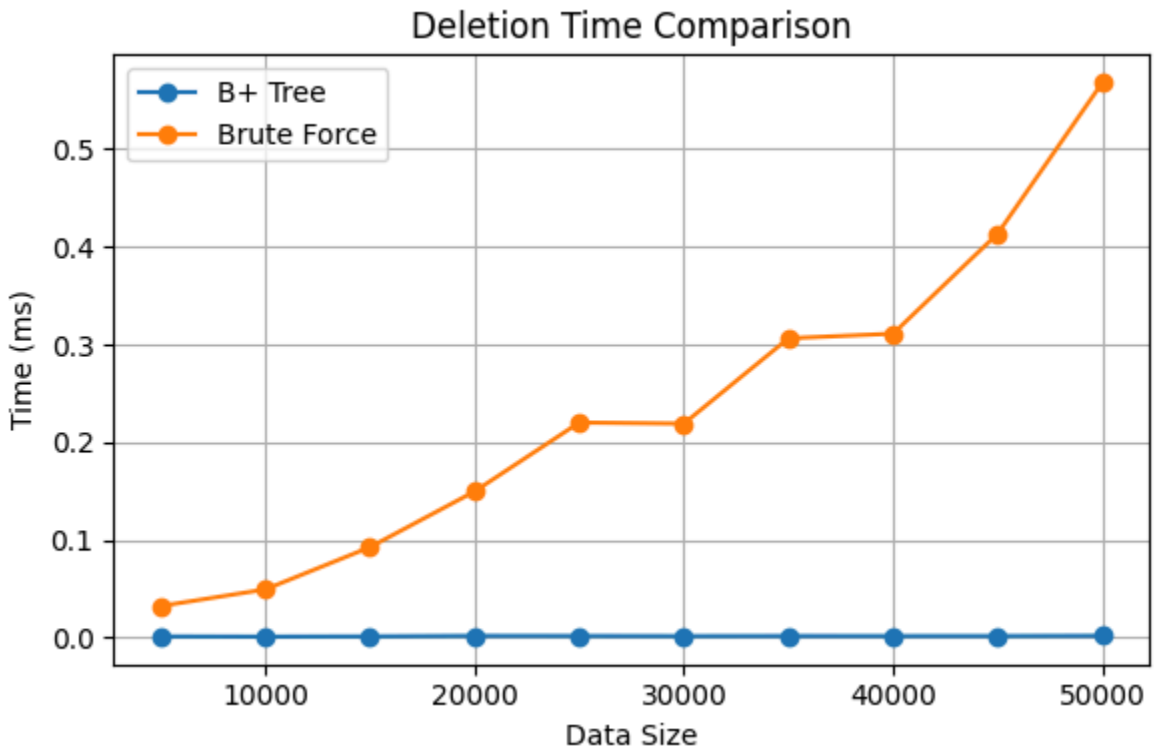
- The B+ Tree search time remains extremely low and almost constant across all dataset sizes. This indicates efficient lookup performance due to its hierarchical structure, which enables search operations in logarithmic time O(logn)
- The Brute Force search time increases significantly as the dataset size grows. Since it performs a linear scan of the list, its time complexity is O(n)
- $O(n)$, leading to slower performance for larger datasets.
- The gap between the two approaches widens rapidly with increasing data size, clearly demonstrating the scalability advantage of the B+ Tree.
- The B+ Tree curve appears almost flat, showing high stability and consistency, whereas the Brute Force curve shows a steep upward trend, indicating poor scalability.

Key Insight

The B+ Tree significantly outperforms the Brute Force approach in search operations due to its logarithmic time complexity. This makes it highly suitable for applications requiring fast data retrieval, especially in large-scale database systems.

3. **Graphical analysis of Deletion Time Performance Comparison:**

The deletion performance graph compares the time required to remove records from the B+ Tree and the Brute Force data structure as the dataset size increases.



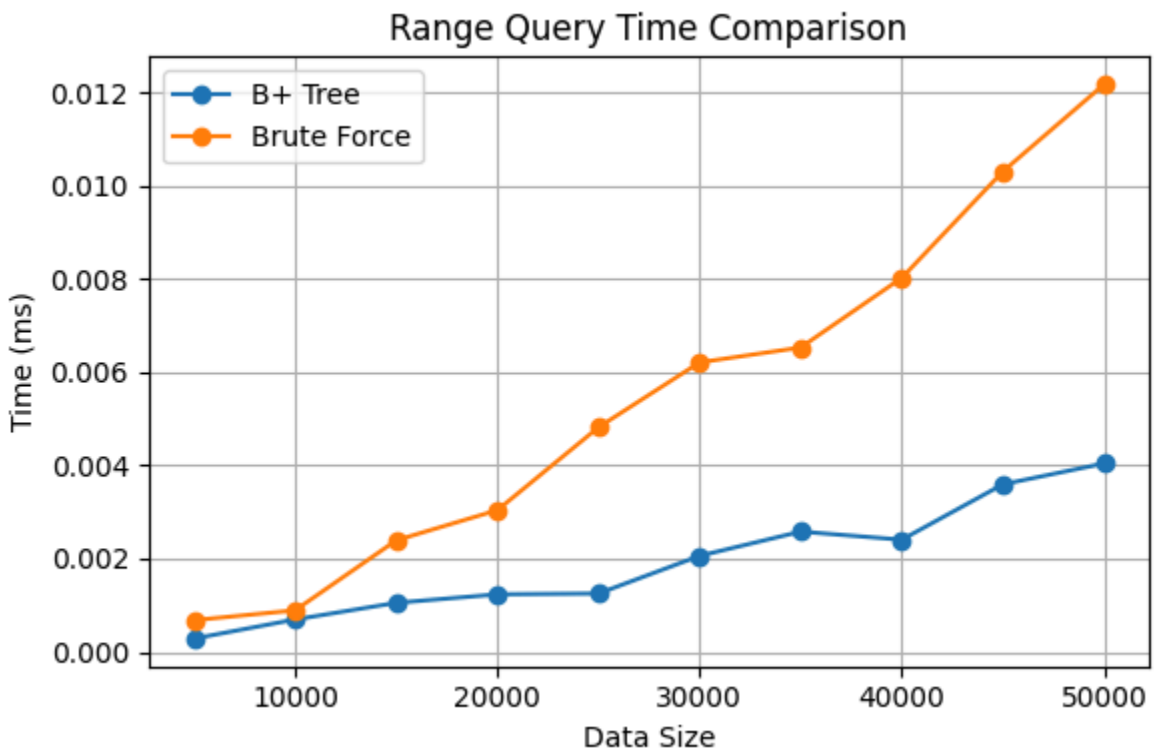
From the graph, it is observed that:

- The B+ Tree deletion time remains very low and nearly constant across all dataset sizes. This indicates efficient deletion due to its balanced tree structure, which ensures logarithmic time complexity O(logn).
- The Brute Force deletion time increases steadily as the dataset size grows. Since deletion requires first locating the element and then removing it from the list, the overall complexity becomes O(n).
- $O(n)$, leading to higher execution time.
- The gap between the B+ Tree and Brute Force approaches widens significantly with increasing dataset size, highlighting the scalability advantage of the B+ Tree.
- The B+ Tree curve is almost flat, showing stable and consistent performance, whereas the Brute Force curve shows a sharp upward trend, indicating inefficiency for large datasets.

Key Insight: The B+ Tree provides significantly faster and more scalable deletion performance compared to the Brute Force approach. Its ability to maintain balance through borrowing and merging ensures efficient operations even for large datasets.

4. **Graphical Analysis of Range Query Time Performance Comparison:**

The range query performance graph compares the time required to retrieve records within a specified key range for both the B+ Tree and the Brute Force data structure as the dataset size increases.



From the graph, it is observed that:

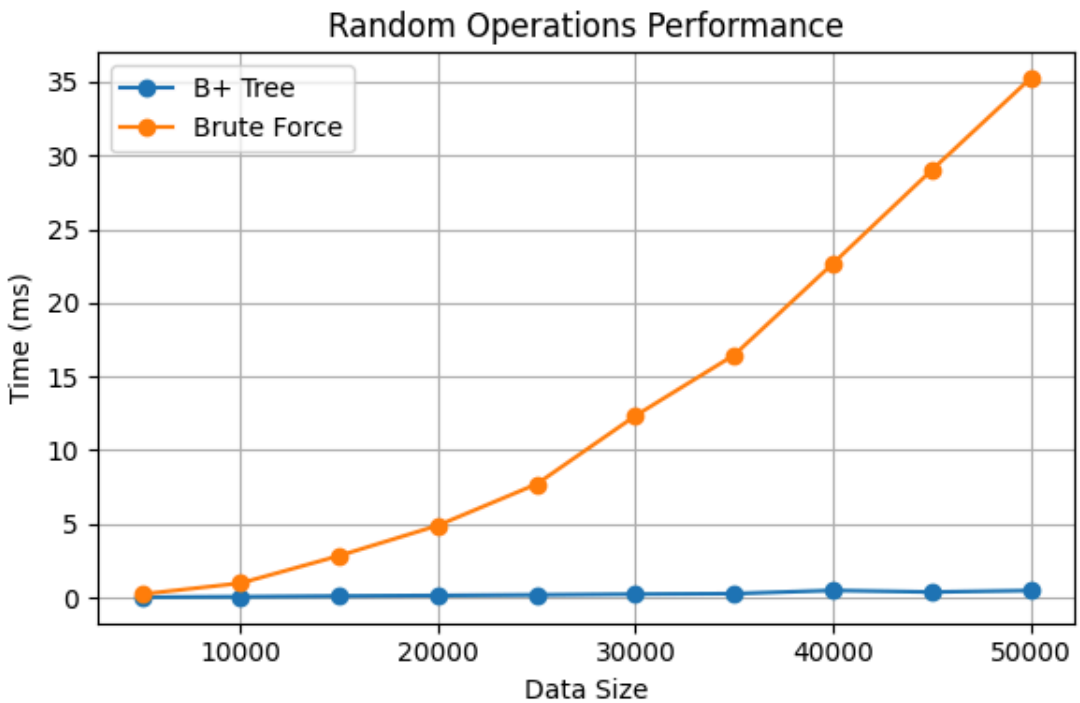
- The B+ Tree consistently performs faster than the Brute Force approach for range queries across all dataset sizes.
- The B+ Tree shows a gradual and controlled increase in execution time as the dataset size grows. This is because it efficiently locates the starting point and then traverses only the relevant records using linked leaf nodes (`next` pointer).
- In contrast, the Brute Force approach shows a steeper increase in execution time, as it scans the entire dataset to find elements within the given range, resulting in linear time complexity $O(n)$.
- The performance gap between the two approaches widens with increasing dataset size, clearly demonstrating the advantage of structured indexing.
- The B+ Tree curve is smoother and more stable, while the Brute Force curve shows a sharper upward trend, indicating reduced efficiency at scale.

Key Insight:

The B+ Tree significantly improves range query performance by leveraging its leaf-level linkage, allowing sequential access to only relevant data. This makes it highly efficient for range-based queries compared to the Brute Force approach, which requires a full dataset scan.

5. **Graphical Analysis of Random Operations Performance Comparison:**

The random operations performance graph compares the execution time of mixed operations (insertion, search, and deletion) for both the B+ Tree and the Brute Force data structure as the dataset size increases.



From the graph, it is observed that:

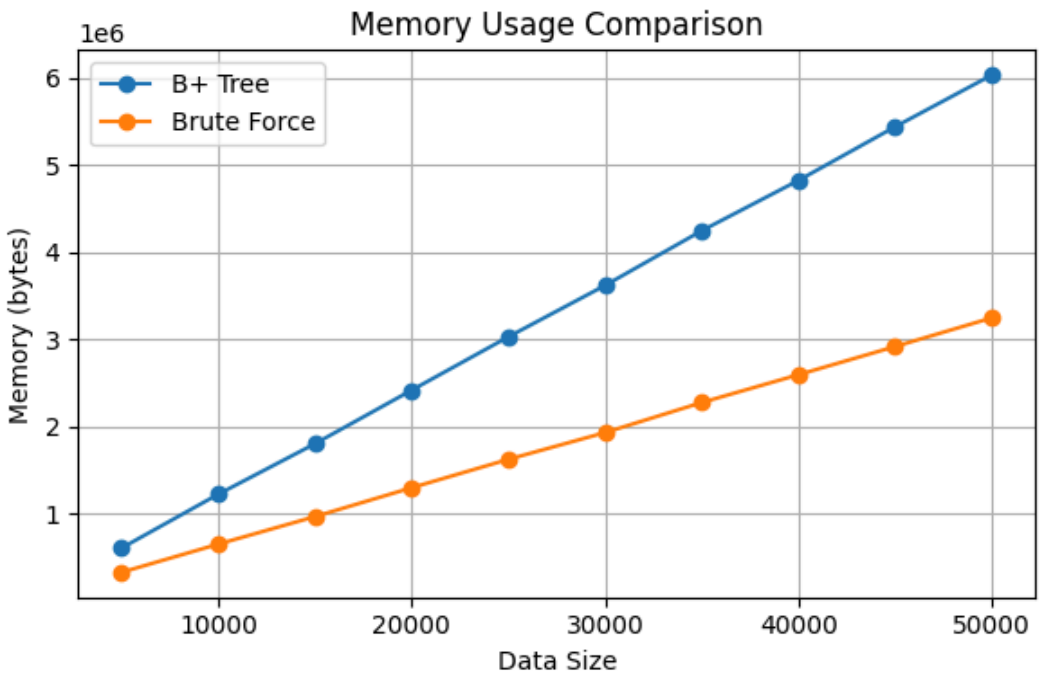
- The B+ Tree maintains consistently low execution time even as the dataset size increases. This indicates its ability to efficiently handle mixed workloads due to its balanced and indexed structure.
- The Brute Force approach shows a sharp and exponential increase in execution time as the dataset size grows. Since each operation may require a full linear scan, the cumulative cost becomes very high under mixed workloads.
- The performance gap between the two approaches widens dramatically with increasing data size, especially beyond 20,000 records.
- At higher dataset sizes, the Brute Force curve rises steeply, indicating poor scalability, whereas the B+ Tree curve remains relatively flat, demonstrating high efficiency and stability.

Key Insight:

This graph clearly demonstrates that the B+ Tree is far more efficient for real-world scenarios involving mixed operations. While individual operations may vary in cost, the overall performance of the B+ Tree remains stable, whereas the Brute Force approach becomes significantly slower due to repeated linear traversals.

6. Graphical Analysis of Memory Usage Performance Comparison:

The memory usage graph compares the amount of memory consumed by the B+ Tree and the Brute Force data structure as the dataset size increases.



From the graph, it is observed that:

- The B+ Tree consistently uses more memory than the Brute Force approach across all dataset sizes.
- The memory consumption of both structures increases linearly with the growth in dataset size, indicating proportional scaling with the number of records.
- The B+ Tree shows a steeper increase in memory usage, as it stores additional structural information such as nodes, child pointers, and leaf node linkages.
- In contrast, the Brute Force approach uses less memory because it stores data in a simple list without any additional indexing or hierarchical structure.

- The gap between the two approaches remains consistent, reflecting the overhead introduced by maintaining the B+ Tree structure.

Key Insight: The B+ Tree trades higher memory usage for significantly improved performance in search, deletion, and range queries. This trade-off is acceptable and commonly adopted in database systems where performance is prioritised over minimal memory usage.

The graphs show that while the Brute Force approach is faster for insertion, the B+ Tree performs significantly better in search, deletion, range queries, and mixed operations. Although it uses more memory, the B+ Tree provides better scalability and consistent performance, making it more suitable for large datasets.

B+ Tree Visualisation

To better understand the structure and behavior of the B+ Tree, visualisations were generated for different dataset sizes. These visualisations illustrate how the tree grows and maintains balance as the number of keys increases.

The following figures represent the B+ Tree structure for:

- A small dataset
- A medium dataset
- A large dataset

Each visualisation highlights internal nodes, leaf nodes, and the linkage between leaf nodes.

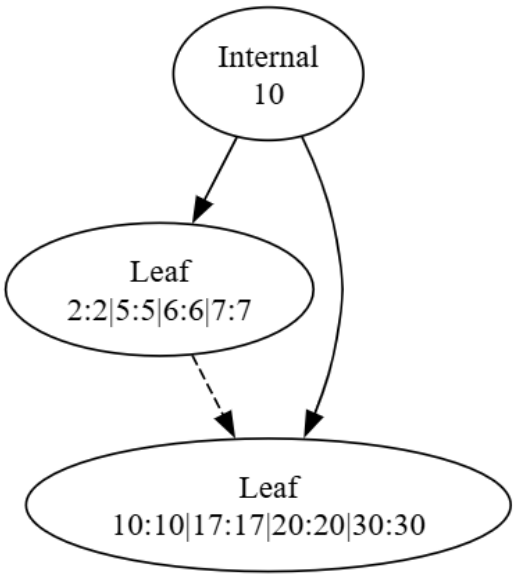
B+ Tree Degree Specification

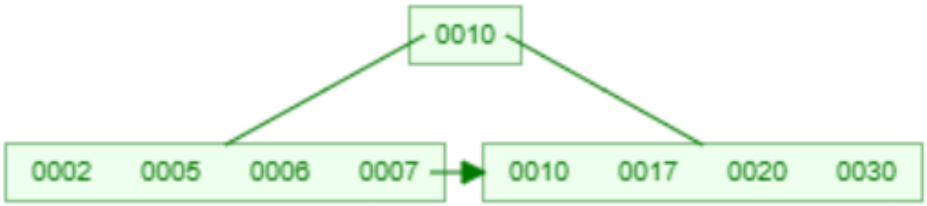
In this implementation, the B+ Tree is constructed with a minimum degree $t=3$. This determines the range of keys that each node can hold.

- Maximum number of keys in a node = $2t-1=5$
- Minimum number of keys in a node (except root) = $t-1=2$
- Maximum number of children = $2t=6$
- Minimum number of children = $t=3$

Thus, each node in the tree contains between 2 and 5 keys, ensuring that the tree remains balanced and supports efficient operations with O(logn)O(\log n)O(logn) time complexity.

1. Small Dataset Visualisation





Reference tree generated using https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html

The figure illustrates a B+ Tree constructed with a minimum degree t=3 using the keys 2, 5, 6, 7, 10, 17, 20, and 30. Conceptually, a B+ Tree organizes data in a hierarchical manner where internal nodes act as index guides, and all actual data is stored only in the leaf nodes.

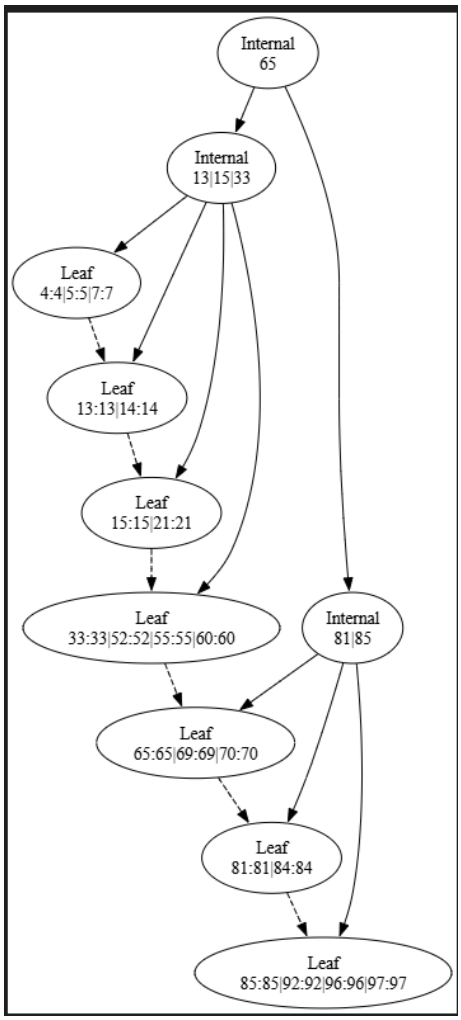
In this example, the root node contains the key 10, which acts as a separator directing keys less than 10 to the left leaf node (2, 5, 6, 7) and keys greater than or equal to 10 to the right leaf node (10, 17, 20, 30). The keys in each leaf node are stored in sorted order.

Additionally, the leaf nodes are connected using a pointer (shown as a dashed link), which enables efficient sequential traversal for range queries. This structure ensures that the tree remains balanced and supports efficient operations with O(logn) time complexity.

2. Medium Dataset Visualisation

The figure represents a B+ Tree constructed from a medium-sized dataset of 20 randomly generated keys (range 1–100) with a minimum degree t=3. As the dataset size increases, the tree expands vertically, introducing multiple levels of internal nodes to maintain balance**.**

Conceptually, the B+ Tree organizes data in a multi-level hierarchical structure, where internal nodes act as index nodes and leaf nodes store the actual data. As the number of keys increases, the tree grows in height to maintain balance



In this visualisation:

- The root node (65) divides the data into two major subtrees, guiding the search process.
- Intermediate internal nodes (13, 15, 33 and 81, 85) further partition the data into smaller ranges.
- The leaf nodes store sorted key-value pairs, such as:
    - 4, 5, 7
    - 13, 14
    - 15, 21
    - 33, 52, 55, 60

    - 65, 69, 70
    - 81, 84
    - 85, 92, 96, 97
- All leaf nodes are connected through sequential pointers (dashed links), allowing efficient traversal for range queries.

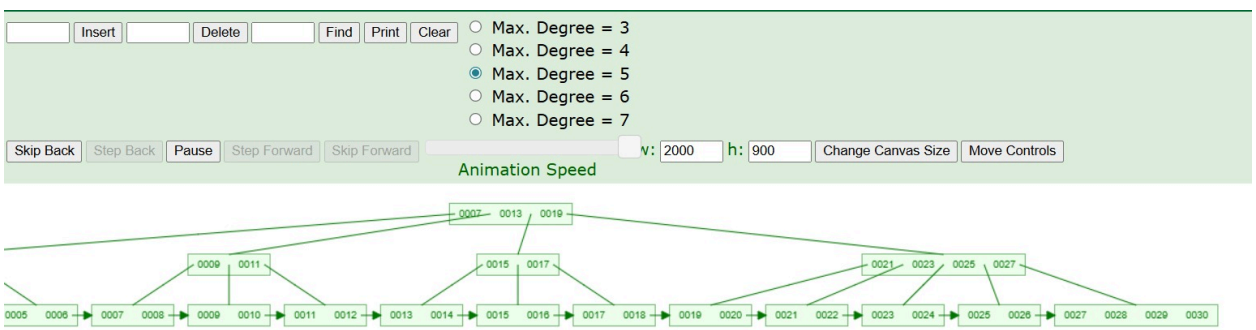
This structure demonstrates how the B+ Tree maintains balance by distributing keys across multiple levels, ensuring that all leaf nodes remain at the same depth. As a result, operations such as search, insertion, and deletion are performed efficiently with

O(logn) time complexity.

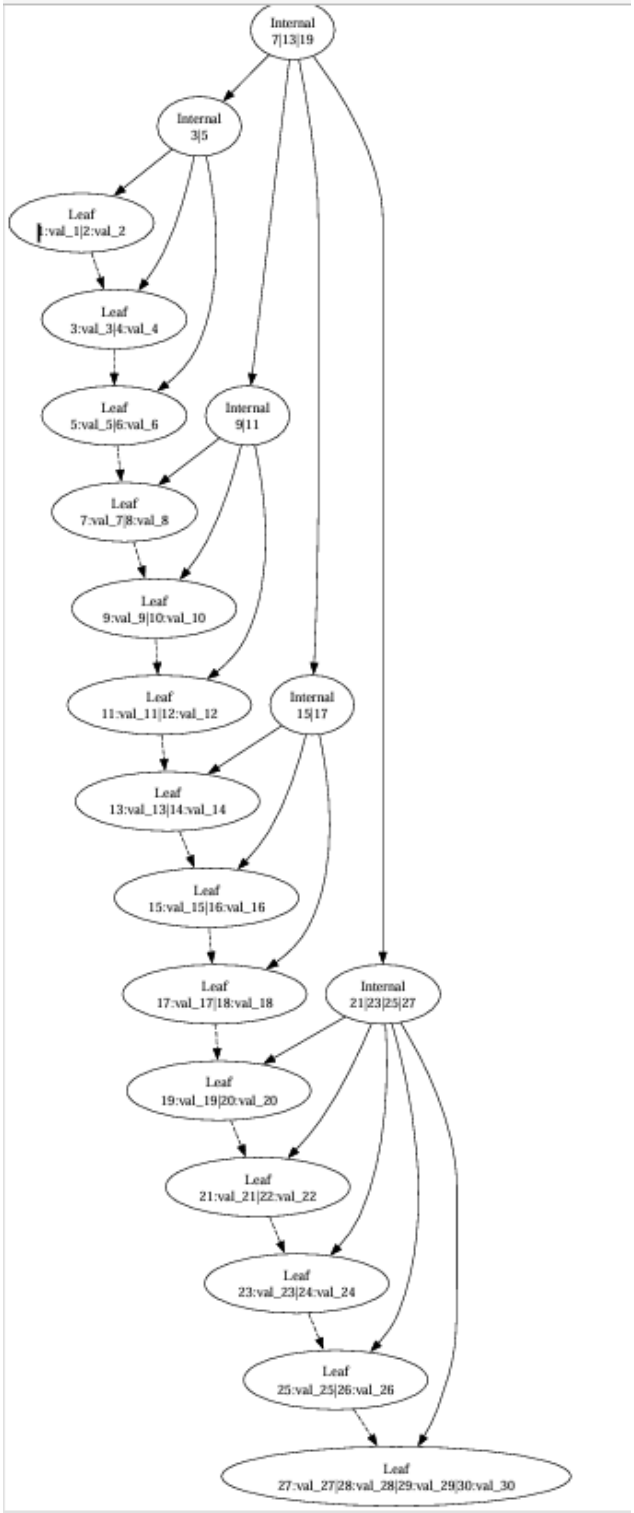
3. <u>Large Dataset Visualisation using B+ tree</u>

The figure represents a B+ Tree constructed using a dataset of 30 sequential keys (1 to 30).

As the number of keys increases, the tree grows into multiple levels while maintaining balance through node splitting and redistribution. Internal nodes act as indexing layers that guide the search path, while all actual data is stored in leaf nodes at the same depth. The leaf nodes are sequentially linked, enabling efficient range queries. This structure ensures scalable and consistent performance with $O(\log n)$ time complexity.



https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html


Internal
7|13|19
Internal
3|5
Leaf
1:val_1|2:val_2
Leaf
3:val_3|4:val_4
Leaf
5:val_5|6:val_6
Internal
9|11
Leaf
7:val_7|8:val_8
Leaf
9:val_9|10:val_10
Leaf
11:val_11|12:val_12
Internal
15|17
Leaf
13:val_13|14:val_14
Leaf
15:val_15|16:val_16
Leaf
17:val_17|18:val_18
Internal
21|23|25|27
Leaf
19:val_19|20:val_20
Leaf
21:val_21|22:val_22
Leaf
23:val_23|24:val_24
Leaf
25:val_25|26:val_26
Leaf
27:val_27|28:val_28|29:val_29|30:val_30

DB Manager Implementation Demonstration

The functionality of the DB Manager was validated through a series of operations demonstrating table management, data manipulation, and error handling

```
1. CREATE TABLES
Created tables: users, orders

2. LIST TABLES
  Tables currently present: ['users', 'orders']

3. GET TABLE AND USE IT
  users[101] = {'id': 101, 'name': 'Alice', 'role': 'admin', 'email': 'alice@example.com', 'created_at': '2026-03-21'}
  users[102] = {'id': 102, 'name': 'Bob', 'role': 'analyst', 'email': 'bob@example.com', 'created_at': '2026-03-21'}
  users range [100, 200] = [(101, {'id': 101, 'name': 'Alice', 'role': 'admin', 'email': 'alice@example.com', 'created_at': '2026-03-21'}), (102, {'id': 102, 'name': 'Bob', 'role': 'an

4. DROP TABLE
  Dropped table: orders
  Tables after drop: ['users']

5. ERROR HANDLING
  Expected get_table error: Table 'orders' does not exist.
  Expected create_table error: Table 'users' already exists.

6. VALIDATION DEMOS (Simulated)
  Expected allowed-value validation error: Column 'role' must be one of ['admin', 'analyst', 'viewer'], got 'manager'
  Expected datatype validation error: Column 'id' expects int, got str
```

These outputs demonstrates the working of the DB Manager, including table creation, data insertion, retrieval, deletion, and error handling. It confirms that the system correctly manages tables and records while enforcing validation and handling invalid operations appropriately.

```
1. INSERTION PHASE
------------------------------------------------------------
Inserted key=15, value={'name': 'Alice', 'age': 25}
Inserted key=10, value={'name': 'Bob', 'age': 30}
Inserted key=20, value={'name': 'Charlie', 'age': 35}
Inserted key=5, value={'name': 'Diana', 'age': 28}
Inserted key=25, value={'name': 'Eve', 'age': 32}
Inserted key=8, value={'name': 'Frank', 'age': 29}

2. SEARCH OPERATIONS
------------------------------------------------------------
  ✓ Found: key=15, value={'name': 'Alice', 'age': 25}
  ✓ Found: key=8, value={'name': 'Frank', 'age': 29}
  ✗ Not found: key=99, value=None

3. UPDATE OPERATIONS
------------------------------------------------------------
  ✓ Updated key=15
    New value: {'name': 'Alice', 'age': 26, 'updated': True}

4. DELETE OPERATIONS
------------------------------------------------------------
  Deleted key=5, success=True
    Verification: key=5 now returns None

5. RANGE QUERY
------------------------------------------------------------
  Query: keys in range [8, 20]
  Results (4 records):
    - key=8: {'name': 'Frank', 'age': 29}
    - key=10: {'name': 'Bob', 'age': 30}
    - key=15: {'name': 'Alice', 'age': 26, 'updated': True}
    - key=20: {'name': 'Charlie', 'age': 35}

6. RETRIEVE ALL RECORDS (IN-ORDER)
------------------------------------------------------------
  Total records: 5
    - key=8: {'name': 'Frank', 'age': 29}
    - key=10: {'name': 'Bob', 'age': 30}
    - key=15: {'name': 'Alice', 'age': 26, 'updated': True}
    - key=20: {'name': 'Charlie', 'age': 35}
    - key=25: {'name': 'Eve', 'age': 32}
```

B+ Tree Operations Demonstration

The implementation was tested using a sequence of operations to verify the correctness and functionality of the B+ Tree.

1. Insertion Phase

Initially, multiple records were inserted into the tree with keys **15, 10, 20, 5, 25, and 8**.
Each record consists of a key and a corresponding value (user details).
The B+ Tree automatically maintains the keys in **sorted order** during insertion while ensuring balance through node splitting when required.

2. Search Operations

Search operations were performed for keys **15, 8, and 99**:

- Keys **15** and **8** were successfully found with their corresponding values.
- Key **99** was not present in the tree, and the search correctly returned **None**.

This demonstrates that the search operation correctly retrieves existing records and handles missing keys.

3. Update Operations

The record with key **15** was updated:

- The value associated with key 15 was modified (age updated to 26 and an additional field added).

This confirms that the update operation correctly locates and modifies existing records.

4. Delete Operations

The key **5** was deleted from the tree:

- The deletion was successful, and a follow-up search confirmed that the key no longer exists.

This shows that deletion is handled correctly while maintaining tree integrity.

5. Range Query

A range query was performed for keys between **8 and 20**:

- The result returned all keys within the range: **8, 10, 15, and 20**.

This demonstrates the efficiency of the B+ Tree in performing range queries using **linked leaf nodes**, without scanning the entire dataset.

6. Retrieval of All Records

All records were retrieved in sorted order:

- The output shows keys in ascending order: **8, 10, 15, 20, 25**.

This confirms that the B+ Tree maintains sorted data and supports efficient in-order traversal.

The demonstration verifies that the B+ Tree correctly supports all core operations — insertion, search, update, deletion, and range queries — while maintaining sorted order and balanced structure.

Conclusion:

This assignment demonstrates that while simple linear approaches are easy to implement, they do not scale efficiently with increasing data size. The B+ Tree, by maintaining a balanced and indexed structure, ensures efficient performance across all major operations, especially search, deletion, and range queries.

The results clearly show that investing in a structured indexing mechanism significantly improves overall system efficiency, even at the cost of slightly higher insertion time and memory usage. Hence, B+ Trees provide a practical and scalable solution for data management compared to brute-force methods.

Team Members and Contributions

Members Name	Contribution
Thipparapu Rushitha	Implemented the DB Manager and Table modules, handling table management, CRUD operations, and system integration. Also contributed to report documentation.
Sailaja Mudu	Implemented the B+ Tree data structure, including core operations such as insertion, deletion, search, and range queries. Also contributed to report documentation and participated in the video presentation.
Vadithya Harsha Vardhan Nayak	Implemented the Brute Force database and developed the performance analysis module for benchmarking and comparison. Also contributed to report documentation and participated in the video presentation.
Devvrat Hans	Module B
Nitin Bansal	Module B

Tools and External Resources Used:

- Python: Core implementation
- Graphviz: Tree visualisation
- Matplotlib: Performance graphs
- Pandas: Result organisation
- Random (Python library): Dataset generation
- Jupyter Notebook: Development and reporting
- GitHub: Version control
- YouTube and documentation: Reference for B+ Tree concepts